

Module 4

Q. Explain Different Load Estimation and Process Transfer Policies Used by Load Balancing Algorithms

=>

1. Load balancing is the process of distributing work evenly among processors or computers in a distributed system.
2. Load balancing improves system performance and avoids overloading a single processor.
3. Load balancing algorithms use different policies to decide when and how to transfer tasks.
4. Two important policies are load estimation policy and process transfer policy.

Load Estimation Policy

1. Load estimation policy is used to determine the current load on a processor or node.
2. The load may be measured using CPU usage, memory usage, queue length, or number of processes.
3. This policy helps identify whether a node is overloaded or underloaded.
4. Correct load estimation is important for effective load balancing.

Methods of Load Estimation

1. CPU Queue Length

1. CPU queue length means the number of processes waiting for CPU execution.
2. A large queue length indicates that the processor is heavily loaded.
3. A small queue length indicates that the processor is lightly loaded.
4. This method is simple and commonly used.

2. CPU Utilization

1. CPU utilization measures how much CPU time is being used.
2. High CPU utilization means the processor is busy.
3. Low CPU utilization means the processor is free for more work.
4. This method gives a direct idea about processor usage.

3. Memory Utilization

1. Memory utilization measures how much main memory is occupied.
2. High memory usage indicates a heavy load on the node.
3. Low memory usage indicates that more processes can be accepted.
4. This method is useful when processes require large memory.

4. Number of Running Processes

1. The number of running processes can also be used to estimate load.
2. More processes indicate higher load.
3. Fewer processes indicate lower load.
4. This method is easy to calculate.

5. Response Time

1. Response time means the time taken by a processor to complete a task.
2. Long response time indicates that the system is overloaded.
3. Short response time indicates that the system is lightly loaded.
4. This method helps evaluate system performance directly.

Process Transfer Policy

1. Process transfer policy decides when a process should be moved from one processor to another.
2. The policy helps reduce the load on overloaded processors.
3. It also ensures that underloaded processors are utilized properly.

Types of Process Transfer Policies

1. Sender-Initiated Policy

1. In sender-initiated policy, the overloaded processor starts the transfer process.
2. The overloaded processor searches for a lightly loaded processor.
3. After finding a suitable node, it transfers some of its tasks.
4. This policy works well when most processors are lightly loaded.
5. An example is a busy server sending some jobs to another idle server.

2. Receiver-Initiated Policy

1. In receiver-initiated policy, the underloaded processor starts the transfer process.
2. The underloaded processor searches for overloaded processors.
3. It requests work from heavily loaded nodes.
4. This policy works well when most processors are overloaded.
5. An example is an idle processor asking another busy processor for tasks.

3. Symmetric Policy

1. Symmetric policy combines both sender-initiated and receiver-initiated approaches.
2. Both overloaded and underloaded processors can start the transfer process.
3. This policy provides better flexibility and efficiency.
4. Symmetric policy is suitable for systems with changing workloads.

Process Selection Policy

1. Process selection policy decides which process should be moved to another processor.
2. Small and independent processes are preferred for transfer.
3. Processes that require fewer resources are easier to move.
4. Processes with less communication overhead are also preferred.
5. Large processes are usually not transferred because migration cost is high.

Location Policy

1. Location policy decides the destination processor where the process should be transferred.
2. The algorithm searches for a lightly loaded processor.
3. The selected processor should have enough resources to execute the process.
4. Proper location policy improves the efficiency of load balancing.

Priority Assignment Policy

1. Priority assignment policy decides the importance of transferred and local processes.
2. The system should ensure that transferred processes are not ignored.
3. Local processes and transferred processes should get fair CPU time.
4. Proper priority management avoids starvation.

Q. Explain Key Features of Global Scheduling Algorithm

=>

Introduction

1. Global scheduling is a scheduling technique used in distributed systems to assign tasks to processors across the entire system.
2. Global scheduling considers all processors and tasks together instead of scheduling tasks locally on one processor.
3. The main aim of global scheduling is to improve resource utilization and system performance.
4. Global scheduling helps balance workload among processors.

Key Features of Global Scheduling Algorithm

1. System-Wide Scheduling

1. Global scheduling makes decisions by considering all processors in the distributed system.
2. Tasks can be assigned to any processor in the network.
3. This feature improves overall efficiency of the system.

2. Load Balancing

1. Global scheduling distributes tasks evenly among processors.
2. It prevents some processors from becoming overloaded while others remain idle.
3. Proper load balancing improves processor utilization.
4. An example is transferring tasks from a busy server to an idle server.

3. Process Migration

1. Global scheduling supports migration of processes from one processor to another.
2. Process migration helps reduce overload on busy nodes.
3. Migration improves system performance and response time.
4. An example is moving a heavy task from one server to another server with low load.

4. Dynamic Decision Making

1. Global scheduling algorithms make decisions according to current system conditions.
2. The scheduler monitors processor load, memory usage, and network conditions.
3. Dynamic scheduling is more flexible than static scheduling.
4. It can adapt to changing workloads in the system.

5. Resource Sharing

1. Global scheduling allows processors to share system resources efficiently.
2. It ensures better use of CPU, memory, and storage resources.

3. Resource sharing improves the performance of distributed systems.

6. Scalability

1. Global scheduling algorithms can support a large number of processors and tasks.
2. The algorithm continues to work effectively even when the system grows.
3. Scalability is important in cloud computing and large distributed systems.

7. Fault Tolerance

1. Global scheduling algorithms can handle processor failures.
2. If one processor fails, tasks can be reassigned to other processors.
3. Fault tolerance increases system reliability and availability.

8. Efficient Communication

1. Global scheduling tries to reduce communication overhead between processors.
2. Tasks are assigned in a way that minimizes network traffic.
3. Efficient communication improves system performance.

9. Improved Throughput

1. Throughput means the number of tasks completed in a given time.
2. Global scheduling increases throughput by using all processors effectively.
3. More tasks can be completed in less time.

10. Reduced Response Time

1. Response time means the time taken to complete a task after receiving it.
2. Global scheduling reduces waiting time for tasks.
3. Faster task execution improves user satisfaction.
4. An example is faster processing of requests in cloud servers.

11. Priority Handling

1. Global scheduling algorithms can assign priority to important tasks.
2. High-priority tasks are executed before low-priority tasks.
3. This feature is useful in real-time systems.

12. Better Resource Utilization

1. Global scheduling ensures that all processors are used properly.
2. Idle processors can be assigned pending tasks.
3. Better resource utilization reduces wastage of system resources.

Q. Explain Code Migration and its Techniques, Issues of Code Migration, and the Role of Process-to-Resource and Resource-to-Machine Binding in Code Migration

=>

Code Migration

1. Code migration is the process of moving a program, process, or code segment from one machine to another machine in a distributed system.
2. Code migration is used to improve performance, reduce load, and use system resources efficiently.
3. In code migration, the execution of the process may continue on the new machine after transfer.
4. Code migration is useful in distributed computing, mobile agents, and cloud computing.
5. An example of code migration is moving a process from an overloaded server to an idle server.

Need for Code Migration

1. Code migration helps in load balancing by shifting tasks from busy machines to less busy machines.
2. Code migration improves system performance by executing code closer to required resources.
3. Code migration helps reduce network communication cost.
4. Code migration supports fault tolerance because processes can move away from failed machines.
5. Code migration allows better utilization of system resources.

Techniques of Code Migration

1. Weak Mobility

1. In weak mobility, only the code and some initialization data are transferred to another machine.
2. The execution state of the process is not transferred.
3. The process starts execution from the beginning on the new machine.
4. Weak mobility is simple to implement.
5. An example is transferring a Java applet to another system for execution.

2. Strong Mobility

1. In strong mobility, both the code and the current execution state are transferred to another machine.
2. The process resumes execution from the exact point where it stopped.
3. Strong mobility is more complex than weak mobility.
4. Strong mobility is useful when long-running tasks need to continue on another machine.
5. An example is migrating a virtual machine from one server to another without stopping execution.

Types of Code Migration

1. Sender-Initiated Migration

1. In sender-initiated migration, the source machine decides to move the code to another machine.
2. This is usually done when the source machine is overloaded.
3. The source machine selects a suitable destination machine for migration.

2. Receiver-Initiated Migration

1. In receiver-initiated migration, the destination machine requests work from another machine.
2. This happens when the destination machine is idle or underloaded.
3. The destination machine asks busy machines to transfer processes.

Issues in Code Migration

1. State Transfer Problem

1. During migration, the process state must be transferred correctly.
2. The state includes variable values, program counter, and memory contents.
3. Incorrect state transfer may cause the process to fail.

2. Resource Availability Problem

1. The destination machine may not have all the resources needed by the process.
2. Some files, devices, or software libraries may not be available.
3. This may affect execution after migration.

3. Communication Problem

1. The migrated process may need to communicate with other processes still running on the original machine.
2. Maintaining communication links after migration is difficult.
3. Communication delay may increase after migration.

4. Security Problem

1. Code migration can introduce security risks in distributed systems.
2. A malicious process may damage the destination machine.
3. Sensitive data may also be exposed during migration.

5. Heterogeneity Problem

1. Source and destination machines may have different hardware and operating systems.
2. The migrated code may not run correctly on a different platform.
3. Compatibility issues may arise during migration.

6. Performance Overhead

1. Migration itself requires time and network bandwidth.
2. Large processes may take longer to transfer.
3. Too much migration may reduce system performance instead of improving it.

Process-to-Resource Binding in Code Migration

1. Process-to-resource binding means the relationship between a process and the resources it needs.
2. Some resources are attached closely to the process and cannot be moved easily.
3. Resources may include files, printers, databases, or memory.
4. There are three types of process-to-resource binding.

a. By Identifier

1. The resource can be accessed using its name or identifier.
2. The process can continue to use the resource after migration.
3. An example is accessing a remote file using its file name.

b. By Value

1. The actual value of the resource is copied along with the process.
2. The process uses the copied resource on the destination machine.
3. An example is copying a local file to the new machine.

c. By Type

1. The process only requires a resource of a particular type.
2. Any available resource of that type can be used on the destination machine.
3. An example is using any available printer on the destination machine.

Resource-to-Machine Binding in Code Migration

1. Resource-to-machine binding means the relationship between a resource and the machine where it is located.
2. Some resources are fixed to a particular machine and cannot be moved.
3. Other resources can be transferred or accessed remotely.

Types of Resource-to-Machine Binding

1. Unattached Resources

1. Unattached resources can be moved easily from one machine to another.
2. These resources are not permanently tied to any machine.
3. An example is a software file or a data file.

2. Fastened Resources

1. Fastened resources are attached to a machine but can sometimes be moved with extra effort.
2. These resources are more difficult to transfer than unattached resources.
3. An example is a local database stored on a server.

3. Fixed Resources

1. Fixed resources cannot be moved from their original machine.
2. These resources are permanently attached to a machine.
3. An example is a printer, camera, or hardware device connected to a specific computer.